



DevOps

Dr. John-Paul Ore

Housekeeping

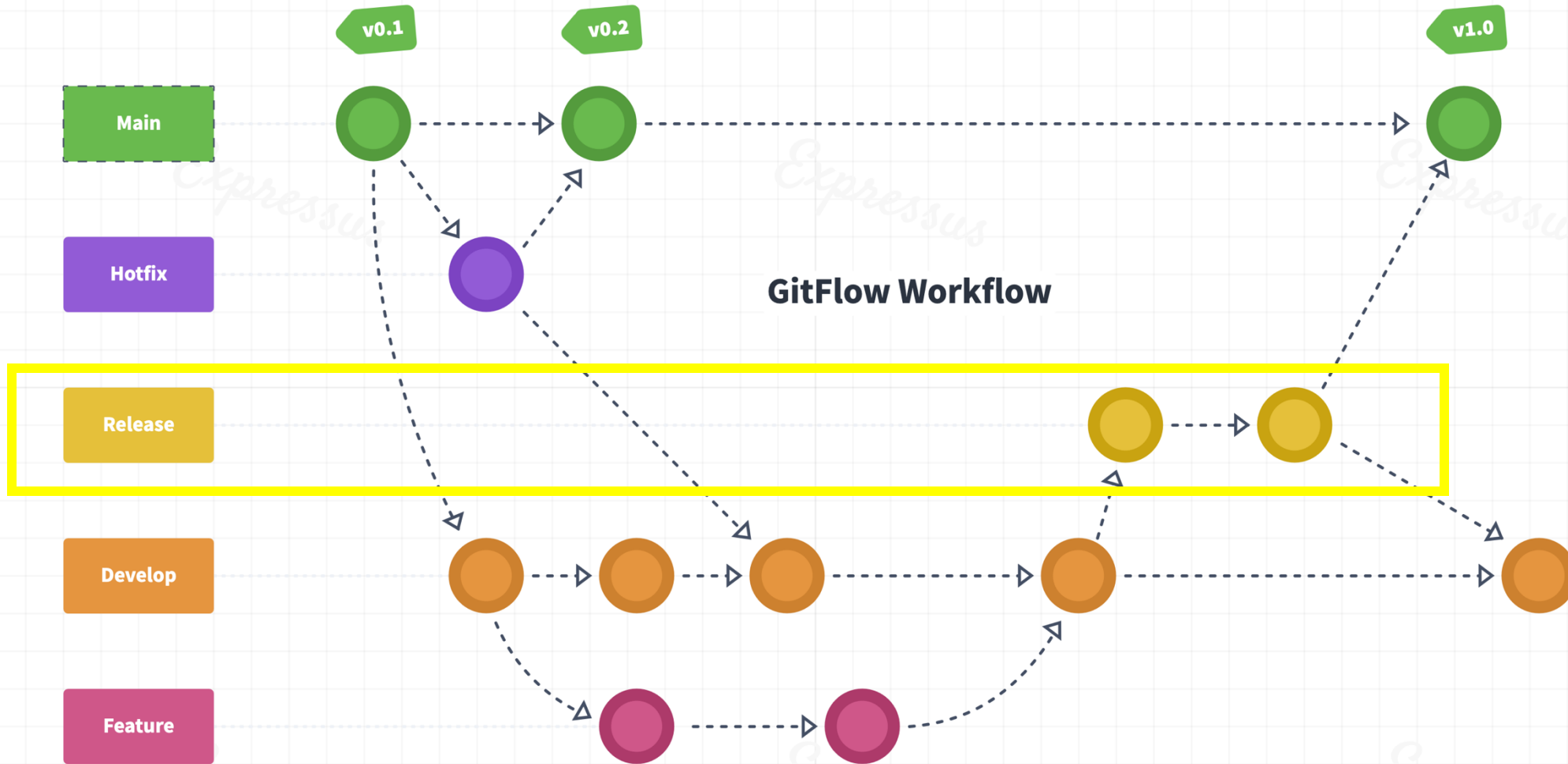
- Workshop 1 is Due this Tuesday, Aug 26th AOE.
- TA Office Hours (Kaushik spillal2@ncsu.edu) Tuesday's 1:30-2:30pm [by zoom](#) or in-person by arrangement.
- For Wednesday's (8/27) workshop, bring a computer that can run Docker containers.

Today's Learning Objectives

- Roles in DevOps Teams
- *"Diff Time"*
- *Review*: REST APIs
- Continuous Integration / Continuous Deployment (CI/CD)
- DevOps & CI/CD

A Git-based Development Workflow a.k.a “Git Flow”

Also called Branch based development



Continuous Integration (CI)

- “Continuous” is imprecise, because code changes happen at discrete moments in time, called “**Diff Time.**”
- Tools like git include Pull Requests (PRs).
- PRs can include 1 or more code commits .
- Frequently putting code changes together a.k.a “integrating” (in multi-person teams) makes it easier to identify / debug problems.
- Smaller changes ➔ Easier debugging.

An SE event: “Diff Time”

- Critical moment in software engineering process.
- This is when code peer review happens (a.k.a “Modern Code Review”)¹
- Increasingly, this is when a kind of testing called “Static Analysis” happens.²

1. “Modern code review: a case study at Google” (2018)

Caitlin Sadowski, Emma Söderberg, Luke Church, Michal Sipko, and Alberto Bacchelli

2. “Scaling static analyses at Facebook” (2019)

Dino Distefano, Manuel Fähndrich, Francesco Logozzo, Peter W. O'Hearn

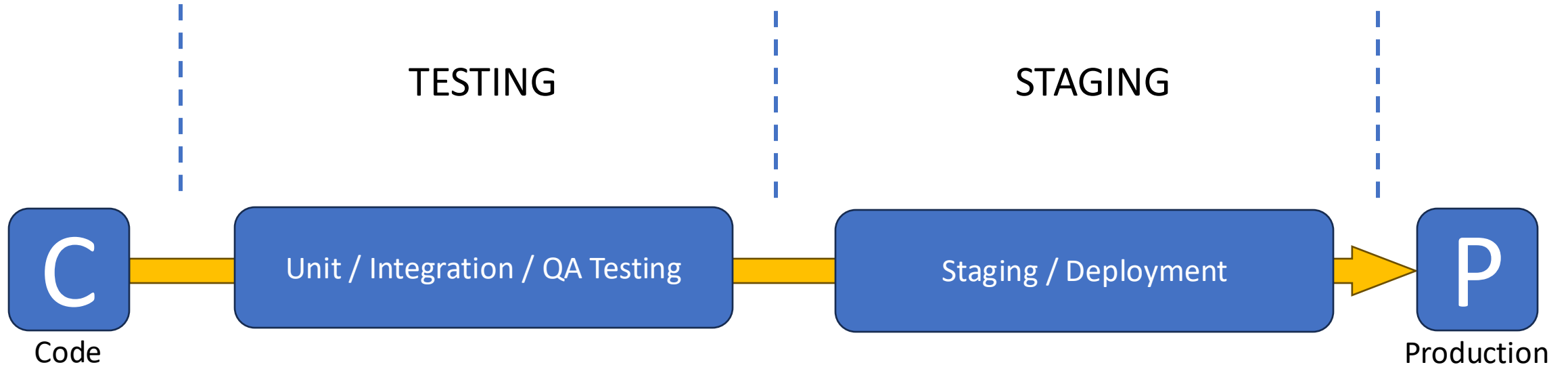
Continuous Deployment

- “Continuous” is imprecise, because updates to a production system under active development updates happens at intervals.
- The CD Part of CI/CD (*“See Eye, See Dee”*)

DevOps Pipeline



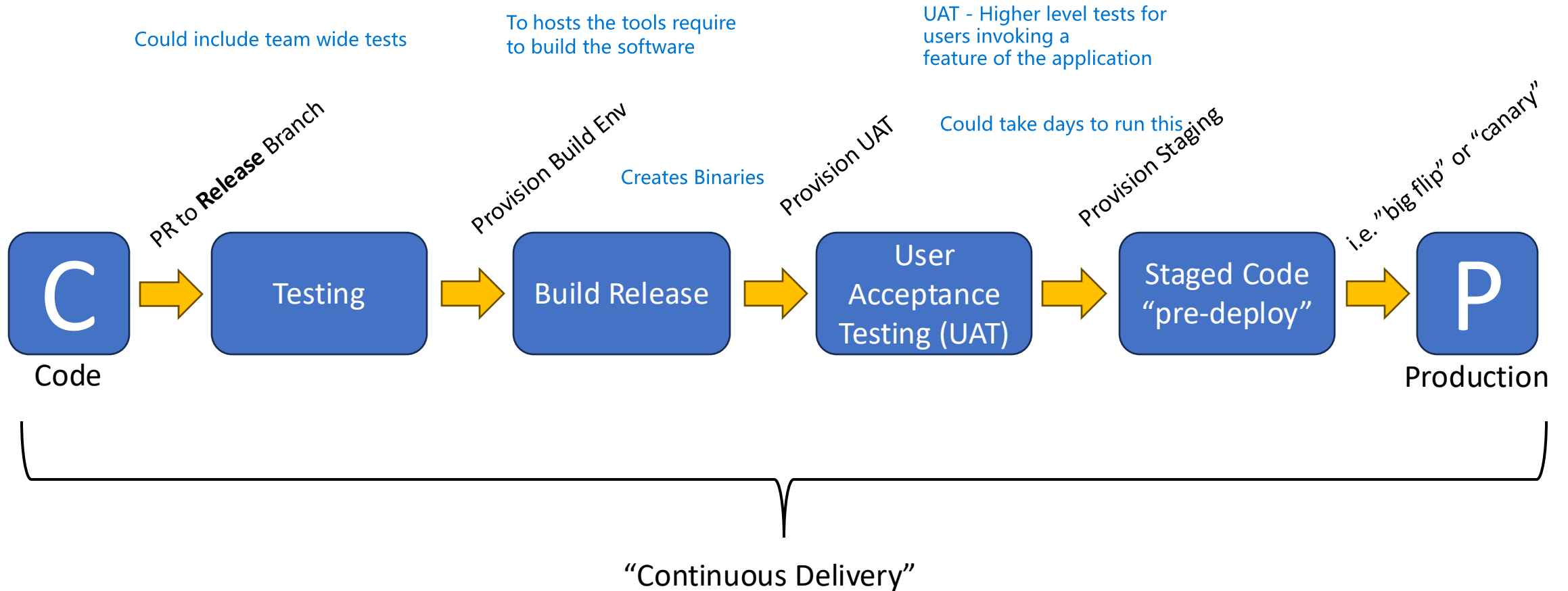
DevOps Pipeline



- 4 Different Environments
- Dev
 - Testing
 - Staging
 - Production

An example DevOps pipeline

Canary Releasing - putting the code out for a smaller group of users



DevOps “Events” or “Triggers”

- Code Commit
- Pull Request to special branch
- Automated Testing
- Passing tests (a “**quality gate**”) can automatically trigger CI/CD pipeline **promotion** to the next phase of continuous deployment, i.e. the “**staging**” phase.

DevOps Environments

- **Development (Dev)**
- **Testing (Test/QA)**
- **Staging (Staging/UAT)**
- **Production (Prod)**

Other environments:

- **Integration**
- **Disaster Recovery (DR)**

ACTIVITY: Think-Pair-Share

- 1) **Think** by yourself quietly about the activity's prompt: (usually 1 min)
- 2) **Pair** up with a few people around you and discusses what you thought about.
- 3) **Share** with the class.

Prompt:

What could go wrong in a DevOps pipeline?

Insufficient Testin
- coverage

Inadequate Resources ,
Resource Constraints

Time Consumin

Environment Mismatch

DevOps team size: The “Two Large Pizzas” Rule

- At Amazon, development teams should be no larger than what can be fed from two pizzas. This depends on the pizza.



Teams roles in the DevOps process

- **Service Owner** : Outside coordination. Maintains and communicates vision of the service.
- **Gatekeeper** : Responsible for decisions to move to next stage in deployment pipeline.
(decisions might be automated).
- **Release Engineer / Coordinator** : Responsible for entire release process, including approving late-breaking additions, back-out decision, and referring bug severity disputes.
- **DevOps Engineer** : Responsible for care & feeding of tools used throughout the DevOps toolchain.
- **Reliability Engineer** : Responsible for time period immediately after deployment.

Some “DevOps” job postings

S Software Engineer-Kubernetes, DevOps & Automation

SAS • Cary HQ, North Carolina • 21 days ago

S DevOps Engineer

SAGA Diagnostics • Morrisville, NC • 21 days ago

⚡ Quick Ap

C DevOps Engineer

Cardinal Health

Cardinal Health • Raleigh, NC • 17 days ago • 📶 Remote ↕

💰 \$79,700–\$119,490 Per Year ↕

V Senior DevOps Engineer

VAST Data • Raleigh, NC • 30+ days ago

🔗 Ap

H Lead Enterprise AI/ML DevOps Engineer

Humana • Raleigh, NC • 4 days ago • 📶 Remote ↕

💰 \$129,300–\$177,800 Per Year ↕

S DevOps Engineer

Syntricate Technologies Inc • Morrisville, NC • 30+ days ago

🔗 A

Some “DevOps” job postings

Description

Required Skills:

- Strong hands-on experience with Ansible for automation, orchestration, and configuration management.
- Strong Python programming skills, including experience in developing automation scripts, APIs, and integrations.
- Expertise in writing Ansible playbooks, roles, and custom modules.
- Proficient in Linux/Unix system administration, including networking, security, and shell scripting.



DevOps Engineer - SRE CloudOps

Bristol Myers Squibb

Princeton, NJ

 Apply



Summary:

BMS is seeking a result-oriented DevOps Engineer. This position is part of our SRE Cloud Operations group based in Princeton, NJ.

The ideal candidate will have expertise in cloud-based services with one or more cloud providers, Azure preferred, AWS or GCP. Must have scripting background and ability to automate repetitive tasks. As a DevOps Engineer you will work with development teams, service providers, business partners, cyber security, and senior members of our team to deliver cloud solutions and manage operations in support of BMS business applications.

Responsibilities will include incorporating software engineering aspects and applying them to infrastructure and operations problems. Focus will be on cloud operations and Site

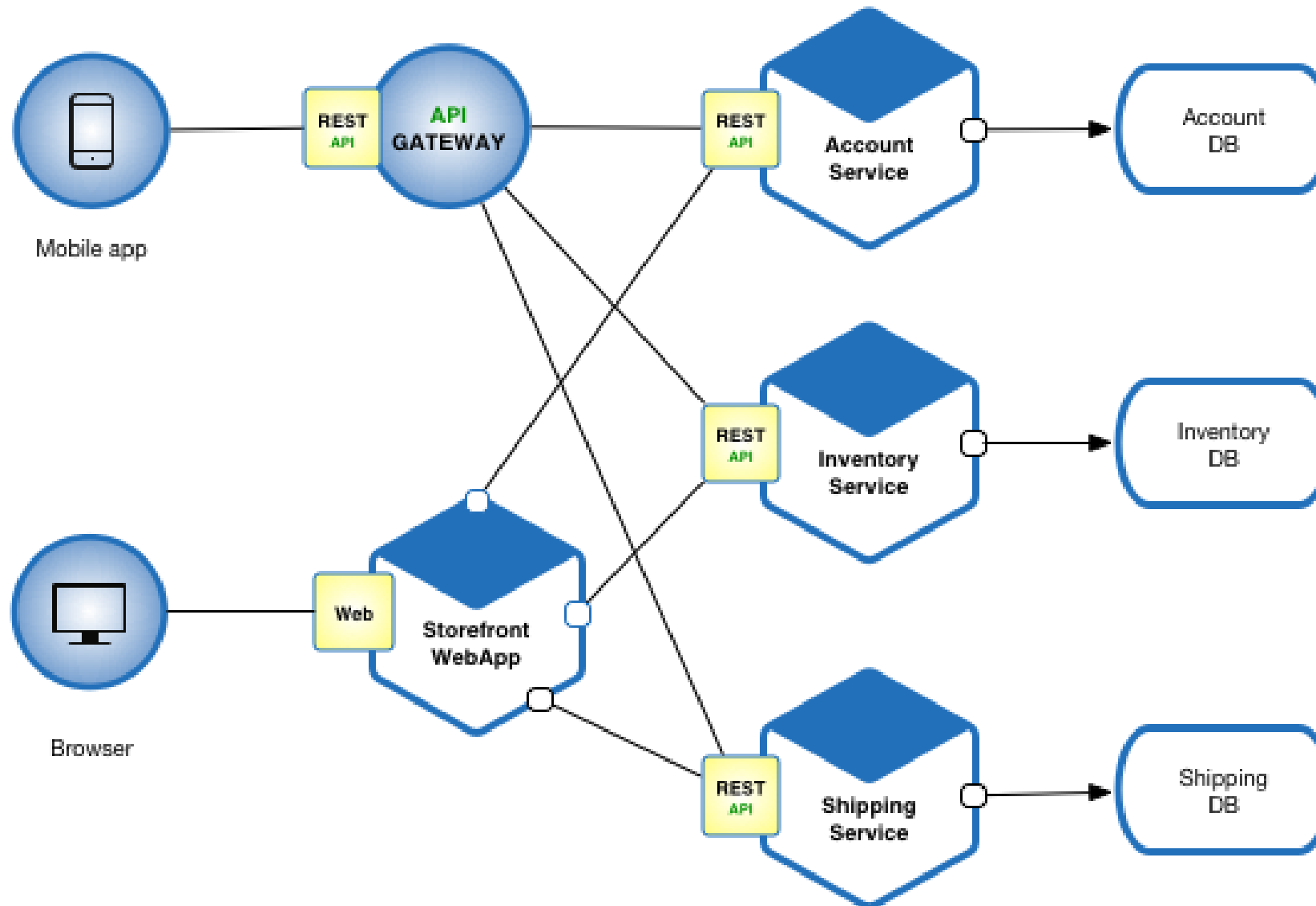
Reliability Engineering (SRE) practices, including automation and optimization of mission and business critical applications and ensuring the highest levels of availability, performance, and security.

“...incorporating software engineering aspects and applying them to...**operations problems**”

Development is organized into “Services”

- A service is exposed in over a public and well-defined interface, i.e. a set of REST APIs.
- Each team is responsible for a separate service.
- If the application is architected into smaller services, then you can maintain services separately.
- When a system is composed of many independent services, it is sometimes called a “**microservices architecture**.”
- *“A distributed system is one in which the failure of a computer you didn’t even know existed can render your own computer unusable.”*
-- **Leslie Lamport**

Example Microservice Architecture



More on this later in the semester.

Src: <https://microservices.io>

Terms

- Provisioning
- Toolchain



REST APIs



Heroes of Software Engineering

Roy Thomas Fielding (1965-)

PhD Thesis (2000)

“Architectural Styles and the Design of Network-based Software Architectures”

Chapter 5:

Representational State Transfer

REST



Img:uci



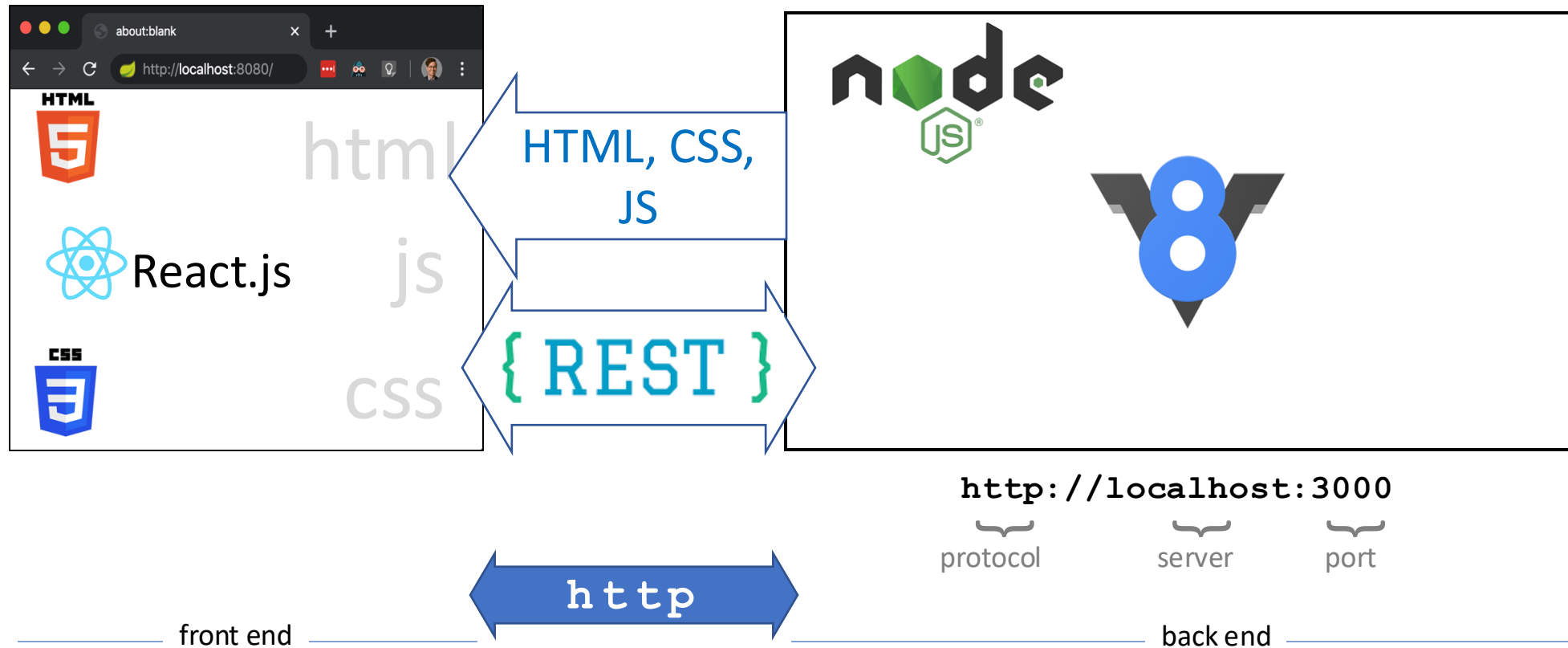
Img:adobe

A Sample High-Level App Architecture

Transferring data
in a stateless fashion

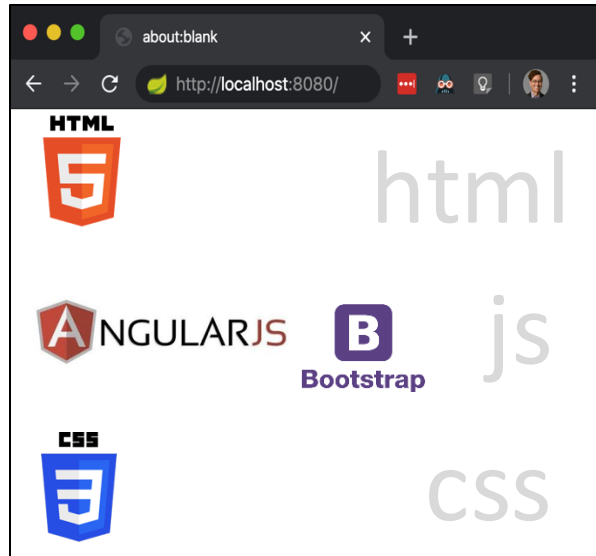
Browser

App



REST returns JSON

Browser



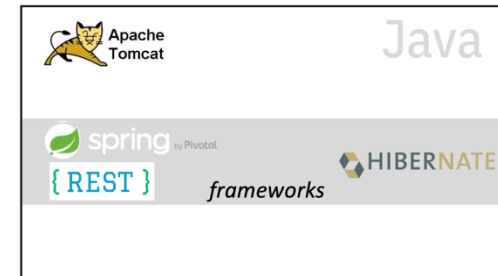
GET http://localhost:3000/api/v1/recipes

http

```
{
  "id": 89,
  "name": "Recipe1",
  "price": 10,
  "coffee": 1,
  "milk": 1,
  "sugar": 1,
  "chocolate": 1
},
{
  "id": 90,
  "name": "coffee",
  "price": 60,
  "coffee": 5,
  "milk": 3,
  "sugar": 7,
  "chocolate": 2
}
```

App

CoffeeMaker, iTrust



Database



What are API's?

- **Problem:** *How can my software talk to other software?*
- **Solution:** ***APIs** Application Programming Interface*
- **Example:** { REST }

REST APIs

- Provide an interface for clients (typically web systems) to interact with back end models, controllers, and services
- Consume and produce JSON
- Actions use HTTP requests, like GET, POST, DELETE, PUT

HTTP Verb	Action	Example	Result
GET	Retrieve record(s)	GET /recipes/	Retrieves all recipes
DELETE	Delete record(s)	DELETE /recipes/1	Deletes recipe with id of 1
POST	Create record	POST /recipes/	Creates a new recipe
PUT	Update record	PUT /recipes/3	Updates recipe 3

NOTE: You'll need to send along some extra JSON data for the last two

REST APIs – Background

- REST = Representational State Transfer
- *Software Architectural Style*
 - Roy T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Ph.D Dissertation. Chapter 5
 - Set of properties and constraints
 - Each API may be implemented in a slightly different way
- Joshua Bloch – How to Design a Good API and Why it Matters
<https://www.youtube.com/watch?v=heh4OeB9A-c&t=348s>

Example APIs

- Facebook Graph API <https://developers.facebook.com/docs/graph-api>
- Twitter <https://developer.twitter.com/en/docs/twitter-api>
- TikTok <https://developers.tiktok.com/>
- Reddit <https://www.reddit.com/dev/api/>
- Twitch <https://dev.twitch.tv/docs/api/>

API General Principles (1)

- Do one thing well
 - You can never remove it (["Hyrum's Law"](#))
 - If it's hard to name, that's a bad sign (more later)
- Implementation should not impact API
 - Don't "leak" implementation details
 - Façade pattern
- Minimize accessibility of EVERYTHING
 - Information hiding
 - Modules used independently

API General Principles (2)

- Names Matter
 - Names should be self-explanatory
 - Consistent naming – words mean the same
- Don't make the client do something the API could *easily* do
 - Reduce need for boilerplate code

RESTful APIs

Architectural Properties

- Performance – appropriate component interactions
- Scalability – support large numbers of components and interactions among components
- Simplicity of interfaces
- Modifiability of components
- Visibility of communication between components by service agents
- Portability of components by moving program code with the data
- Reliability when failures within components, connectors, or data

RESTful APIs

- Architectural Constraints
 - Client-server
 - Stateless
 - Cacheable
 - Layered system
 - Uniform Interface
 - Identification of resources
 - Manipulation of resources through these representations
 - Self-descriptive messages

Pragmatic REST API Design

- Audience: developer
 - Goal: Make developer as successful as possible
- Nouns = Resources (top level object)
 - Collections
 - Element of collection
- Verbs = Operations on collections and element
 - Use HTTP Verbs
 - POST (create)
 - GET (read)
 - PUT (update)
 - DELETE (delete)

/dogs

/dogs/1234

Example REST API Interactions

Resource	POST (create)	GET (read)	PUT (update)	DELETE (delete)
/dogs	Create a new dog	List dogs	Bulk update dogs	Delete all dogs
/dogs/1234	Error	Show Bo	If exists, update Bo If not, error	Delete Bo

Resource Associations

- Handling associations adds another level to the URL
 - Once have primary key for one level (an element) you don't need to include levels above it
 - Pattern
- Example: Dogs have owners
 - Get all the dogs for an owner
 - `/resource/identifier/resource`
 - Create a new dog for an owner

GET /owners/5678/dogs

POST /owners/5678/dogs

Parameters: States and Attributes

- States and resource attributes shouldn't be handled as associations
 - Add them behind the HTTP question mark

```
GET /dogs?color=red&state=running&location=park
```

Handling Errors

- Errors provide context to client misuse of the black-box behind the API
 - Use HTTP status codes
 - Everything worked – success
 - 200 – OK
 - 201 – Created
 - The application did something wrong – client error
 - 400 – Bad Request
 - The API did something wrong – server error
 - 500 – Internal Server Error
 - Add additional codes as appropriate
 - Returned messages should be verbose

Other Considerations

- Versioning – APIs should be versioned
 - Make version mandatory in URL/header
- Pagination
 - Provide way to limit number of responses
 - Offset and limit
- Partial Response
 - Let clients request a subset of information about a resource
- Authentication – Use common libraries (OAuth)

Responses without Resources

- Goes back to making sure that you aren't making your client do things the API could do

- Common actions

```
/convert?from=EUR&to=CN¥&amount=100
```

- Use verbs

- Clearly define actions vs. resources in documentation
 - Consistency matters
 - Search

- Global
 - Scoped

```
/search?q=fluffy+fur
```

```
/owners/5678/dogs?q=fluffy+fur
```

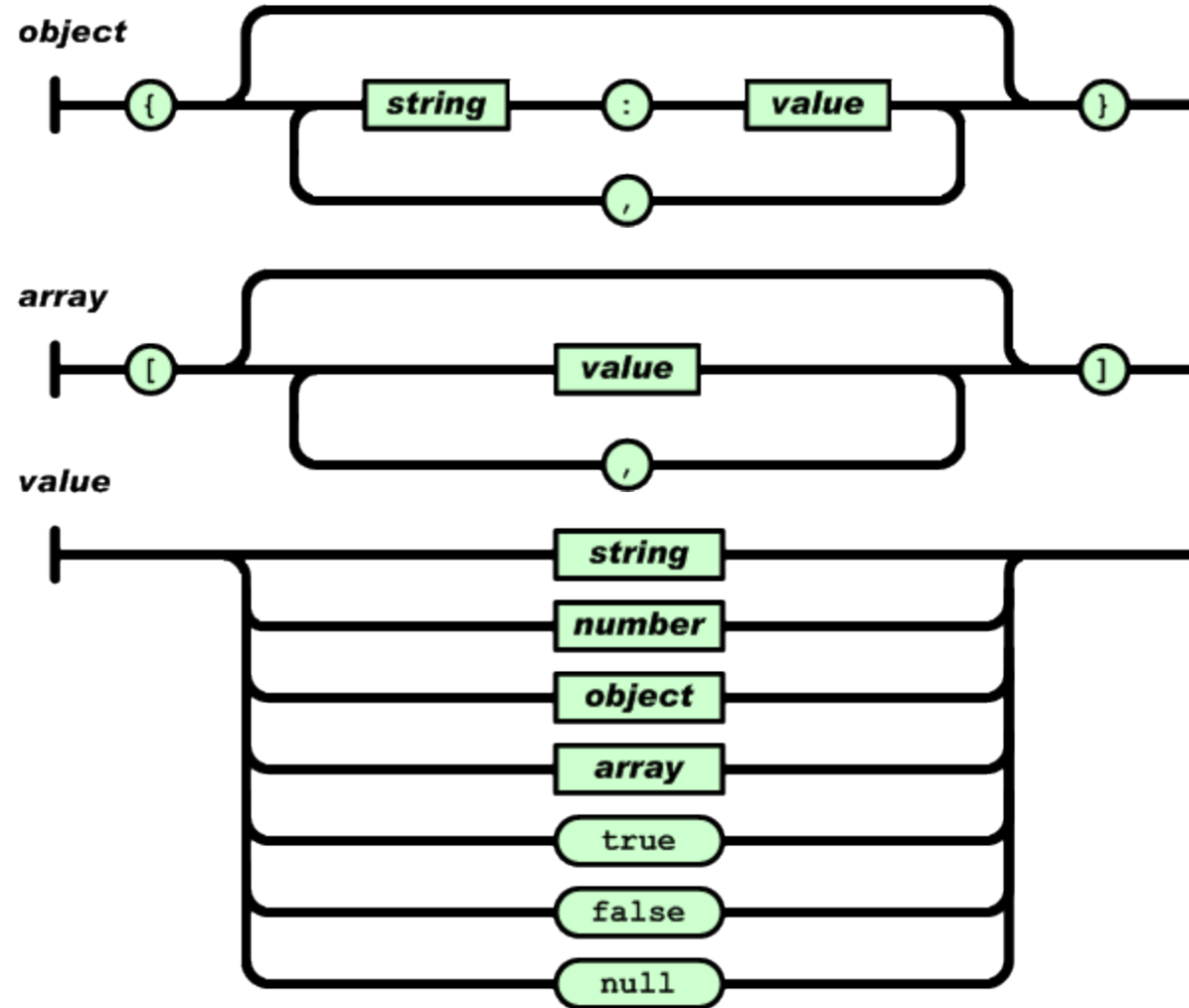

Response Formats

- XML (Extensible Markup Language)
 - Store and transport data
 - Schema defines the tags that describe the data
 - Libraries to help with binding XML to objects ([JAXB](#))
- JSON (JavaScript Object Notation)
 - Lightweight data-interchange format
 - Collection of name/value pairs (object)
 - Ordered list of values (array)

```
var myObject = JSON.parse(response);
```

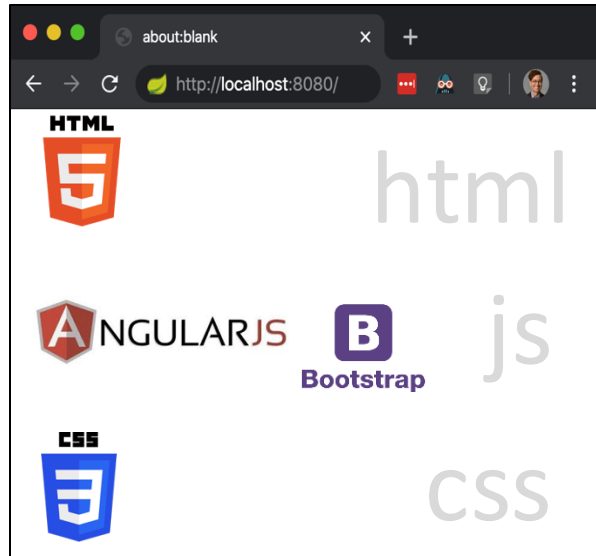
JSON Structures

- Object
- Array
- Value



REST returns JSON

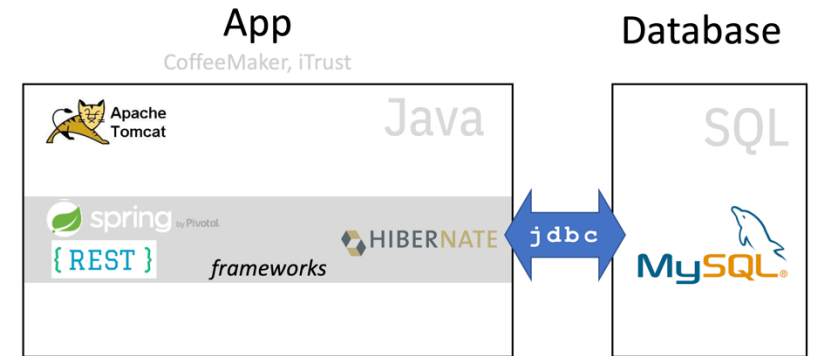
Browser



GET <http://localhost:3000/api/v1/recipes>



```
{
  "id": 89,
  "name": "Recipe1",
  "price": 10,
  "coffee": 1,
  "milk": 1,
  "sugar": 1,
  "chocolate": 1
},
{
  "id": 90,
  "name": "coffee",
  "price": 60,
  "coffee": 5,
  "milk": 3,
  "sugar": 7,
  "chocolate": 2
}
```



REST API Design Documentation

REST API endpoint (relative URL)

Name of function

`getSatisfactionSurveysForPatient()` - (GET /satisfactionsurvey/patient/) - This API endpoint will return all surveys that have been filled in for a given patient. This endpoint will be used on the 'Past Office Visits' page so we can present users with surveys that they have already filled in.

Description

Example - using our data listed above, if we used the API endpoint to get surveys filled in by `user1`, we would call this endpoint with parameter `1` and it would return:

Data details

```
surveys: {  
  {  
    'surveyID': '5',  
    'patient': 'knight',  
    'hcp': 'doc',  
    'minsInWaitingRoom': '5',  
    'minsWaitedForHcp': '10',  
    'officeVisitSatisfaction': '3',  
    'treatmentSatisfaction': '5',  
    'comments': ''  
  }  
}
```

Cont

